```
#include <LoRa.h>
#include <TinyGPS++.h>
#include <SoftwareSerial.h>
#include <Adafruit_MPU6050.h>
#include <math.h>
#include <EEPROM.h>

// CONFIGURATION
#define ANIMAL_ID "A01"

// DEBUG
#define DEBUG 0
#define PRINTE 0 //for current feature important debug only
//#define debug 1

#define LORA_FREQUENCY 433E6
#define GPS_RX 4
#define GPS_TX 3
#define MAX_FENCE_POINTS 4
#define LIVE_INTERVAL 5000UL

//EEPROM
#define EEPROM_PACKET_SEQ_ADDR 0

// HARDWARE OBJECTS
SoftwareSerial gpsSerial(GPS_RX, GPS_TX);
TinyGPSPlus gps;
Adafruit_MPU6050 mpu;

// BATTERY
#define BATTERY_INTERVAL 30000UL

float batteryVoltage = 0.0;
unsigned long lastBatteryRead = 0;

// LIVE MODE
bool liveMode = false;
unsigned long lastLiveSend = 0;

bool pollWaitingForSlot = false;
unsigned long pollReceivedTime = 0;

// MOVEMENT SETTINGS
#define ACCEL_THRESHOLD 0.35
```

```
#define GYRO_THRESHOLD 0.15
#define MOVEMENT_CONFIRM 5
#define STATIONARY_CONFIRM 15
#define FILTER_ALPHA 0.20
#define MPU_INTERVAL 50UL

// MOVEMENT
unsigned long lastMPURead = 0;
float filteredAcceleration = 9.81;
bool animalMoving = false;
byte movementCount = 0;
byte stationaryCount = 0;


// GEOFENCE
struct FencePoint
{
  float lat;
  float lon;
};
FencePoint fence[MAX_FENCE_POINTS];
byte fenceCount = 0;

// GEOFENCE STATE
bool fenceStateKnown = false;
bool animalInside = false;

// ALERT STATE
#define ALERT_BACKOFF_BASE      500UL
#define ALERT_BACKOFF_STEP      3000UL
#define ALERT_BACKOFF_RANDOM    500UL

bool alertPending = false;
bool alertWaitingAck = false;
unsigned long alertSendTime = 0;
uint8_t alertRetryCount = 0;
const unsigned long ALERT_ACK_TIMEOUT = 4000;
const uint8_t MAX_ALERT_RETRIES = 4;
unsigned long alertRetryTime = 0;
bool alertWaitingBackoff = false;

// CURRENT ALERT TRANSACTION
unsigned long activeAlertPacketID = 0;
bool activeAlertExit = false;
```

```
bool activeAlertValid = false;

// MULTIHOP NETWORK
#define MAX_HOPS 1
unsigned long packetSequence = 0;

// CONDITIONAL RELAY
#define RELAY_WAIT_TIME 3500UL
#define POLL_RELAY_WAIT_TIME 8000UL

bool relayPending = false;

String pendingRelayPacket = "";
String pendingRelaySource = "";
String pendingRelayDestination = "";
String pendingRelayPayload = "";

unsigned long relayStartTime = 0;
unsigned long currentCommandPacketID = 0;

#define GPS_SLOT_FRAME 30UL

#define A01_DATA_START   0UL
#define A01_DATA_END     6UL

#define A01_RELAY_START  7UL
#define A01_RELAY_END    14UL

#define A02_DATA_START   15UL
#define A02_DATA_END     21UL

#define A02_RELAY_START  22UL
#define A02_RELAY_END    29UL

unsigned long lastDataFrame = 0;
unsigned long lastRelayFrame = 0;

// DUPLICATE PACKET MEMORY
#define RECENT_PACKET_COUNT 10
String recentPackets[RECENT_PACKET_COUNT];
byte recentPacketIndex = 0;

void clearPendingRelay()
{
```

```
  relayPending = false;

  pendingRelayPacket = "";
  pendingRelaySource = "";
  pendingRelayDestination = "";
  pendingRelayPayload = "";

  relayStartTime = 0;
}

void scheduleRelay(
  String packet,
  String source,
  String destination,
  String payload
)
{
  // Only one pending relay for now
  if (relayPending)
  {
    return;
  }

  pendingRelayPacket = packet;
  pendingRelaySource = source;
  pendingRelayDestination = destination;
  pendingRelayPayload = payload;

  relayPending = true;
  relayStartTime = millis();

#if PRINTE
  Serial.println(F("RELAY SCHEDULED"));
  Serial.print(F("Source: "));
  Serial.println(source);

  Serial.print(F("Destination: "));
  Serial.println(destination);

  Serial.print(F("Waiting for response before forwarding..."));
#endif
}

bool isExpectedRelayResponse(String packet)
```

```
{
  if (!relayPending)
  {
    return false;
  }

  if (!packet.startsWith("MH,"))
  {
    return false;
  }

  int comma1 = packet.indexOf(',');
  int comma2 = packet.indexOf(',', comma1 + 1);
  int comma3 = packet.indexOf(',', comma2 + 1);
  int comma4 = packet.indexOf(',', comma3 + 1);
  int comma5 = packet.indexOf(',', comma4 + 1);
  int comma6 = packet.indexOf(',', comma5 + 1);

  if (comma1 < 0 || comma2 < 0 || comma3 < 0 ||
      comma4 < 0 || comma5 < 0 || comma6 < 0)
  {
    return false;
  }

  String source =
    packet.substring(comma2 + 1, comma3);

  String destination =
    packet.substring(comma3 + 1, comma4);

  String payload =
    packet.substring(comma6 + 1);

  source.trim();
  destination.trim();
  payload.trim();

  // Response must travel in the opposite direction.
  //
  // Pending:
  // BASE -> A01
  //
  // Response:
  // A01 -> BASE
```

```
  if (source != pendingRelayDestination)
  {
    return false;
  }

  if (destination != pendingRelaySource)
  {
    return false;
  }

  // ------------------------------------------------
  // LIVE / FENCE / other command -> ACK
  // ------------------------------------------------

  if (pendingRelayPayload.startsWith("LIVE,") ||
      pendingRelayPayload.startsWith("FENCE,"))
  {
    if (payload.startsWith("ACK,"))
    {
#if PRINTE
      Serial.println(F("MATCHING ACK RESPONSE HEARD"));
#endif
      return true;
    }

    return false;
  }

  // ------------------------------------------------
  // POLL -> DATA
  // ------------------------------------------------

  if (pendingRelayPayload.startsWith("POLL,"))
  {
    if (payload.startsWith("DATA,"))
    {
      // Make sure telemetry belongs to the expected animal.
      //
      // Expected:
      // DATA,A01,...

      int comma1 = payload.indexOf(',');
      int comma2 = payload.indexOf(',', comma1 + 1);
```

```
      if (comma1 < 0 || comma2 < 0)
      {
        return false;
      }

      String dataAnimalID =
        payload.substring(comma1 + 1, comma2);

      dataAnimalID.trim();

      if (dataAnimalID == pendingRelayDestination)
      {
#if PRINTE
        Serial.println(F("MATCHING DATA RESPONSE HEARD"));
#endif
        return true;
      }
    }

    return false;
  }

  return false;
}

void updateConditionalRelay()
{
  if (!relayPending)
  {
    return;
  }

  unsigned long waitTime = RELAY_WAIT_TIME;

  if (pendingRelayPayload.startsWith("POLL,"))
  {
    waitTime = POLL_RELAY_WAIT_TIME;
  }

  if (millis() - relayStartTime < waitTime)
  {
    return;
  }
```

```
  // Relay timeout has expired,
  // but wait for the dedicated GPS relay slot.
  if (!isMyRelaySlotStart())
  {
    return;
  }

#if PRINTE
  Serial.println();
  Serial.println(F("RELAY TIMEOUT"));
  Serial.println(F("No response heard."));
  Serial.println(F("Forwarding stored packet."));
#endif

  // Extract header again

  int comma1 = pendingRelayPacket.indexOf(',');
  int comma2 = pendingRelayPacket.indexOf(',', comma1 + 1);
  int comma3 = pendingRelayPacket.indexOf(',', comma2 + 1);
  int comma4 = pendingRelayPacket.indexOf(',', comma3 + 1);
  int comma5 = pendingRelayPacket.indexOf(',', comma4 + 1);
  int comma6 = pendingRelayPacket.indexOf(',', comma5 + 1);

  if (comma6 < 0)
  {
    clearPendingRelay();
    return;
  }

  String packetID =
    pendingRelayPacket.substring(comma1 + 1, comma2);

  String source =
    pendingRelayPacket.substring(comma2 + 1, comma3);

  String destination =
    pendingRelayPacket.substring(comma3 + 1, comma4);

  String hopText =
    pendingRelayPacket.substring(comma4 + 1, comma5);

  String maxHopText =
    pendingRelayPacket.substring(comma5 + 1, comma6);
```

```
  String payload =
    pendingRelayPacket.substring(comma6 + 1);

  int hop = hopText.toInt();
  int maxHop = maxHopText.toInt();

  // Forward only if hop limit allows

  if (hop < maxHop)
  {
    hop++;

    LoRa.beginPacket();

    LoRa.print(F("MH,"));
    LoRa.print(packetID);
    LoRa.print(',');
    LoRa.print(source);
    LoRa.print(',');
    LoRa.print(destination);
    LoRa.print(',');
    LoRa.print(hop);
    LoRa.print(',');
    LoRa.print(maxHop);
    LoRa.print(',');
    LoRa.print(payload);

    LoRa.endPacket();

    LoRa.receive();
  }

  clearPendingRelay();
}
// START MULTIHOP PACKET
// Format:
// MH,packetID,source,destination,hop,maxHop,
void beginMultiHop(String destination, unsigned long packetID)
{
  LoRa.beginPacket();

  LoRa.print(F("MH,"));
  LoRa.print(packetID);
```

```
  LoRa.print(',');
  LoRa.print(F(ANIMAL_ID));
  LoRa.print(',');
  LoRa.print(destination);
  LoRa.print(F(",0,"));
  LoRa.print(MAX_HOPS);
  LoRa.print(',');
}

// PROCESS MULTIHOP PACKET
// Format:
// MH,packetID,source,destination,hop,maxHop,payload
void processMultiHopPacket(const String &packet)
{
  int comma1 = packet.indexOf(',');
  int comma2 = packet.indexOf(',', comma1 + 1);
  int comma3 = packet.indexOf(',', comma2 + 1);
  int comma4 = packet.indexOf(',', comma3 + 1);
  int comma5 = packet.indexOf(',', comma4 + 1);
  int comma6 = packet.indexOf(',', comma5 + 1);

  if (comma1 < 0 || comma2 < 0 || comma3 < 0 || comma4 < 0 || comma5 < 0 || comma6 < 0)
  {
    return;
  }

  // PARSE HEADER
  String packetID = packet.substring(comma1 + 1, comma2);
  String source = packet.substring(comma2 + 1, comma3);
  String destination = packet.substring(comma3 + 1, comma4);
  String hopText = packet.substring(comma4 + 1, comma5);
  String maxHopText = packet.substring(comma5 + 1, comma6);
  String payload = packet.substring(comma6 + 1);

  packetID.trim();
  source.trim();
  destination.trim();
  int hop = hopText.toInt();
  int maxHop = maxHopText.toInt();

  // IGNORE OUR OWN PACKET
  if (source == ANIMAL_ID)
  {
    return;
```

```
  }

  // CHECK IF THIS PACKET IS THE RESPONSE
  // WE WERE WAITING FOR

  if (isExpectedRelayResponse(packet))
  {
  #if PRINTE
    Serial.println(F("EXPECTED RESPONSE HEARD"));
    Serial.println(F("CANCELLING RELAY"));
  #endif

    clearPendingRelay();

    // Continue processing the packet normally.
  }

  // DUPLICATE DETECTION
  String packetKey = source + "," + packetID;

  if (isDuplicatePacket(packetKey))
  {
#if PRINTE
    Serial.println(F("Duplicate MH packet ignored"));
#endif
    return;
  }

  rememberPacket(packetKey);

#if PRINTE
  Serial.println();
  Serial.println(F("MULTIHOP PACKET RECEIVED"));

  Serial.print(F("Source: "));
  Serial.println(source);

  Serial.print(F("Destination: "));
  Serial.println(destination);

  Serial.print(F("Packet ID: "));
  Serial.println(packetID);

  Serial.print(F("Hop: "));
```

```
  Serial.print(hop);
  Serial.print('/');
  Serial.println(maxHop);

  Serial.print(F("Payload: "));
  Serial.println(payload);
#endif

  // PACKET IS FOR THIS ANIMAL
  if (destination == ANIMAL_ID)
  {
  #if PRINTE
     Serial.println(F("Multihop packet reached destination"));
  #endif

     // ALERT ACK
     if (payload.startsWith("ACK,"))
     {
        processAlertAck(payload);
     }

     // POLL COMMAND
     else if (payload.startsWith("POLL,"))
     {
        receivePollCommand(payload);
     }

     // LIVE COMMAND
     else if (payload.startsWith("LIVE,"))
     {
        currentCommandPacketID = packetID.toInt();
#if DEBUG
    Serial.print(F("SET command ID = "));
    Serial.println(currentCommandPacketID);
#endif

        processLiveCommand(payload);
     }

     // FENCE COMMAND
     else if (payload.startsWith("FENCE,"))
{
    currentCommandPacketID = packetID.toInt();
```

```
#if DEBUG
   Serial.println(F("ABOUT TO CALL processFenceCommand"));
   Serial.print(F("payload length = "));
   Serial.println(payload.length());

   Serial.print(F("payload = ["));
   Serial.print(payload);
   Serial.println(F("]"));

   Serial.print(F("ANIMAL_ID = ["));
   Serial.print(ANIMAL_ID);
   Serial.println(F("]"));
#endif

   processFenceCommand(payload);
}

 #if DEBUG
    else
    {
       Serial.println(F("Unknown multihop payload for this animal"));
    }
 #endif

    // Packet reached its destination.
    // Never forward it further.
    return;
 }

 // PACKET IS NOT FOR US.
 //
 // Do not forward immediately.
 // Store it and wait to see whether
 // direct communication succeeds.

 if (hop < maxHop)
 {
   // ---------------------------------------------
   // DATA SPECIAL CASE
   // ---------------------------------------------
   //
   // DATA is the response to a POLL.
   //
   // Do not put DATA into the normal conditional relay
```

```
    // system because there is currently no confirmation
    // that BASE received the DATA.
    //
    if (payload.startsWith("DATA,"))
    {
  #if DEBUG
      Serial.println(F("DATA heard - conditional relay skipped"));
  #endif
      return;
    }

    // ------------------------------------------------
    // NORMAL CONDITIONAL RELAY
    // ------------------------------------------------

    scheduleRelay(
      packet,
      source,
      destination,
      payload
    );
  }
}

// QUEUE GEOFENCE ALERT
void queueAlert(bool exitAlert)
{
  // CREATE A NEW ALERT TRANSACTION
  packetSequence++;
  savePacketSequence();

  activeAlertPacketID = packetSequence;
  activeAlertExit = exitAlert;
  activeAlertValid = true;
  alertPending = true;
  alertWaitingAck = false;
  alertWaitingBackoff = false;
  alertRetryCount = 0;

#if DEBUG
  Serial.println();
  Serial.println(F("================================"));
  Serial.println(F("GEOFENCE ALERT QUEUED"));
  Serial.print(F("Animal: "));
```

```
  Serial.println(F(ANIMAL_ID));
  Serial.print(F("Packet ID: "));
  Serial.println(activeAlertPacketID);

  if (activeAlertExit)
    Serial.println(F("Type: EXIT"));
  else
    Serial.println(F("Type: RETURN"));

  Serial.println(F("================================"));

#endif
}

// SEND ALERT PACKET
void transmitAlert()
{
  if (!activeAlertValid)
  {
    return;
  }

  // START MULTIHOP HEADER
  beginMultiHop("BASE",activeAlertPacketID);

  // ALERT PAYLOAD
  LoRa.print(F("ALERT,"));
  LoRa.print(F(ANIMAL_ID));
  LoRa.print(',');

  if (activeAlertExit)
    LoRa.print(F("EXIT"));
  else
    LoRa.print(F("RETURN"));

  LoRa.print(',');

  // GPS
  if (gps.location.isValid())
  {
    LoRa.print(gps.location.lat(), 5);
    LoRa.print(',');

    LoRa.print(gps.location.lng(), 5);
```

```
    LoRa.print(',');

    LoRa.print(gps.altitude.meters(), 1);
  }
  else
  {
    LoRa.print(F("0,0,0"));
  }

  // UTC DATE + TIME
  LoRa.print(',');
  printUTCDateTime(LoRa);

  // SEND
  LoRa.endPacket();

  LoRa.receive();
  alertWaitingAck = true;
  alertPending = false;
  alertSendTime = millis();
  alertRetryCount++;

#if DEBUG
  Serial.println();
  Serial.println(F("================================"));
  Serial.println(F("GEOFENCE ALERT SENT"));
  Serial.print(F("Animal: "));
  Serial.println(F(ANIMAL_ID));
  Serial.print(F("Packet ID: "));
  Serial.println(activeAlertPacketID);

  if (activeAlertExit)
    Serial.println(F("Type: EXIT"));
  else
    Serial.println(F("Type: RETURN"));

  Serial.print(F("Attempt: "));
  Serial.println(alertRetryCount);
  Serial.println(F("Waiting for ACK..."));
  Serial.println(F("================================"));

#endif
}
```

```
void processAlertAck(String packet)
{
  if (!packet.startsWith("ACK,"))
  {
    return;
  }

  // Only accept ACK if we are actually waiting for one
  if (!alertWaitingAck)
  {
    return;
  }

  // ACK,A01,ALERT,EXIT,1
  int comma1 = packet.indexOf(',');
  int comma2 = packet.indexOf(',', comma1 + 1);
  int comma3 = packet.indexOf(',', comma2 + 1);
  int comma4 = packet.indexOf(',', comma3 + 1);

  if (comma1 < 0 || comma2 < 0 || comma3 < 0 || comma4 < 0)
  {
    return;
  }

  String animalID =
    packet.substring(comma1 + 1, comma2);

  String command =
    packet.substring(comma2 + 1, comma3);

  String alertType =
    packet.substring(comma3 + 1, comma4);

  String packetIDString =
    packet.substring(comma4 + 1);

  animalID.trim();
  command.trim();
  alertType.trim();
  packetIDString.trim();

  // CHECK ANIMAL
  if (animalID != ANIMAL_ID)
  {
```

```
    return;
  }

  // CHECK COMMAND
  if (command != "ALERT")
  {
    return;
  }

  // CHECK ALERT TYPE
  if (activeAlertExit)
  {
    if (alertType != "EXIT")
    {
      return;
    }
  }
  else
  {
    if (alertType != "RETURN")
    {
      return;
    }
  }

  // CHECK PACKET ID
  unsigned long ackPacketID = packetIDString.toInt();

  if (ackPacketID != activeAlertPacketID)
  {
#if DEBUG
    Serial.println();
    Serial.println(F("================================"));
    Serial.println(F("STALE / WRONG ALERT ACK"));
    Serial.print(F("Expected Packet ID: "));
    Serial.println(activeAlertPacketID);
    Serial.print(F("Received Packet ID: "));
    Serial.println(ackPacketID);
    Serial.println(F("ACK IGNORED"));
    Serial.println(F("================================"));
#endif

    return;
  }
```

```
  // ALERT SUCCESS
  alertWaitingAck = false;
  alertPending = false;
  alertWaitingBackoff = false;
  alertRetryCount = 0;
  activeAlertValid = false;

#if DEBUG
  Serial.println();
  Serial.println(F("================================"));
  Serial.println(F("ALERT ACK RECEIVED"));

  Serial.print(F("Animal: "));
  Serial.println(F(ANIMAL_ID));

  Serial.print(F("Type: "));
  Serial.println(alertType);

  Serial.print(F("Packet ID: "));
  Serial.println(ackPacketID);

  Serial.println(F("Alert successfully delivered."));
  Serial.println(F("================================"));
#endif
}

// ALERT SCHEDULER
void updateAlert()
{
  // Nothing to do
  if (!alertPending && !alertWaitingAck && !alertWaitingBackoff)
  {
    return;
  }

  // Waiting for ACK
  if (alertWaitingAck)
  {
    if (millis() - alertSendTime >= ALERT_ACK_TIMEOUT)
    {
      alertWaitingAck = false;

      if (alertRetryCount < MAX_ALERT_RETRIES)
```

```
      {
        // Collision backoff
        unsigned long backoff = getAlertBackoff();
        alertRetryTime = millis() + backoff;
        alertWaitingBackoff = true;

#if DEBUG
        Serial.println();
        Serial.println(F("ALERT ACK TIMEOUT"));
        Serial.print(F("Backoff: "));
        Serial.print(backoff);
        Serial.println(F(" ms"));
        Serial.println(F("Waiting before retry..."));
#endif
      }
      else
      {
        alertPending = false;
        alertWaitingBackoff = false;
        alertWaitingAck = false;
        activeAlertValid = false;

#if DEBUG
        Serial.println(F("ALERT FAILED"));
        Serial.println(F("Maximum retries reached."));
#endif
      }
    }

    return;
  }

  // Waiting for retry backoff
  if (alertWaitingBackoff)
  {
    if (millis() >= alertRetryTime)
    {
      alertWaitingBackoff = false;
      alertPending = true;

#if DEBUG
      Serial.println();
      Serial.println(F("================================"));
      Serial.println(F("ALERT BACKOFF COMPLETE"));
```

```
      Serial.println(F("Retrying alert..."));
      Serial.println(F("================================"));
#endif
    }

    return;
  }

  // Send pending alert
  if (alertPending)
  {
    transmitAlert();
  }
}

// RECEIVE POLL COMMAND
// Expected:
// POLL,A01
// POLL,A02
// RECEIVE POLL COMMAND

// Expected:
// POLL,A01
// POLL,A02

void receivePollCommand(String packet)
{
  if (!packet.startsWith(F("POLL,")))
  {
    return;
  }

  if (packet.substring(5) != ANIMAL_ID)
  {
    return;
  }

#if DEBUG
  Serial.println();
  Serial.println(F("POLL RECEIVED"));
  Serial.print(F("Animal: "));
  Serial.println(F(ANIMAL_ID));
#endif
```

```
  if (liveMode)
  {
    pollWaitingForSlot = true;
    pollReceivedTime = millis();

#if DEBUG
    Serial.println(F("POLL WAITING FOR DATA SLOT"));
#endif
  }
}

// SEND LIVE TELEMETRY
void sendTelemetry()
{
  // Unique ID for this telemetry packet
  packetSequence++;
  savePacketSequence();

  // Start multihop packet to BASE
  beginMultiHop("BASE", packetSequence);

  // Payload
  LoRa.print(F("DATA,"));
  LoRa.print(F(ANIMAL_ID));
  LoRa.print(',');
  LoRa.print(batteryVoltage, 2);
  LoRa.print(',');

  if (gps.location.isValid())
  {
    LoRa.print(gps.location.lat(), 5);
    LoRa.print(',');

    LoRa.print(gps.location.lng(), 5);
    LoRa.print(',');

    LoRa.print(gps.altitude.meters(), 1);
    LoRa.print(',');

    LoRa.print(gps.satellites.value());
    LoRa.print(',');

    LoRa.print(gps.speed.kmph(), 2);
    LoRa.print(',');
```

```
  }
  else
  {
    LoRa.print(F("0,0,0,0,0,"));
  }

  LoRa.print(animalMoving ? '1' : '0');

  unsigned long txStart = millis();

LoRa.endPacket();

unsigned long txTime = millis() - txStart;

#if debug
Serial.print(F("DATA LoRa TX blocking time: "));
Serial.print(txTime);
Serial.println(F(" ms"));
#endif
  LoRa.receive();

#if DEBUG
  Serial.println(F("Multihop telemetry sent"));
#endif
}

// SEND COMMAND ACK USING MULTIHOP
// Examples:
// MH,...,A01,BASE,...,ACK,A01,LIVE,ON
// MH,...,A01,BASE,...,ACK,A01,FENCE,OK
void sendCommandAck(const char *command, const char *value = "")
{
#if DEBUG
    Serial.print(F("ACK sees command ID = "));
    Serial.println(currentCommandPacketID);
#endif
  // Create unique packet ID for this ACK packet
  packetSequence++;

  savePacketSequence();

  // Start multihop packet
  beginMultiHop("BASE", packetSequence);
```

```
  // ACK payload
  LoRa.print(F("ACK,"));

  LoRa.print(F(ANIMAL_ID));

  LoRa.print(',');

  LoRa.print(command);

  if (value[0] != '\0')
  {
    LoRa.print(',');

    LoRa.print(value);
  }

  // ORIGINAL COMMAND PACKET ID
  LoRa.print(',');

  LoRa.print(currentCommandPacketID);

  // Send packet
  LoRa.endPacket();

  // Return to receive mode
  LoRa.receive();

#if DEBUG

  Serial.print(F("MULTIHOP ACK sent: "));

  Serial.print(ANIMAL_ID);

  Serial.print(F(","));

  Serial.print(command);

  if (value[0] != '\0')
  {
    Serial.print(F(","));

    Serial.print(value);
  }
```

```
  Serial.print(F(" | Command ID: "));

  Serial.print(currentCommandPacketID);

  Serial.print(F(" | ACK Packet ID: "));

  Serial.println(packetSequence);

#endif
}

// PROCESS LIVE COMMAND
// Expected:
// LIVE,A01,ON
// LIVE,A01,OFF
void processLiveCommand(String packet)
{
  if (!isForThisAnimal(packet))
  {
#if DEBUG
    Serial.println(F("LIVE command for another animal"));
#endif
    return;
  }

  int secondComma = packet.indexOf(',', 5);

  if (secondComma < 0)
  {
    return;
  }

  String state = packet.substring(secondComma + 1);
  state.trim();

  // LIVE ON
  if (state == "ON")
  {
    liveMode = true;     // Send telemetry immediately
    lastLiveSend = millis() - LIVE_INTERVAL;
    sendCommandAck("LIVE", "ON");     // Tell base that command was received

#if DEBUG
    Serial.println();
```

```
    Serial.println(F("================================"));
    Serial.println(F("LIVE ON"));
    Serial.print(F("Animal: "));
    Serial.println(ANIMAL_ID);
    Serial.println(F("================================"));
#endif
  }
  // LIVE OFF
  else if (state == "OFF")
  {
    liveMode = false; //Stop live transmission immediately.
    sendCommandAck("LIVE", "OFF");     // Tell base that OFF was received

#if DEBUG
    Serial.println();
    Serial.println(F("================================"));
    Serial.println(F("LIVE OFF"));
    Serial.print(F("Animal: "));
    Serial.println(ANIMAL_ID);
    Serial.println(F("================================"));
#endif
  }
}

void processFenceCommand(const String &packet)
{
  #if DEBUG
  Serial.println(F(">>> processFenceCommand() CALLED"));
  Serial.print(F("FENCE packet = ["));
  Serial.print(packet);
  Serial.println(F("]"));
#endif
  if (!isForThisAnimal(packet))
  {
#if DEBUG
    Serial.println(F("FENCE command for another animal"));
#endif
    return;
  }

  // Find ID and point count
  int comma1 = packet.indexOf(',');
  int comma2 = packet.indexOf(',', comma1 + 1);
  int comma3 = packet.indexOf(',', comma2 + 1);
```

```
  if (comma1 < 0 || comma2 < 0 || comma3 < 0)
  {
#if DEBUG
    Serial.println(F("Invalid FENCE command"));
#endif
    return;
  }

  String countText = packet.substring( comma2 + 1, comma3);
  int newCount = countText.toInt();

  // Validate point count
  if (newCount < 3 || newCount > MAX_FENCE_POINTS)
  {
#if DEBUG
    Serial.println(F("Invalid fence point count"));
#endif
    return;
  }

  // Parse fence points
  // Packet format:FENCE,A02,4,lat,lon,lat,lon,lat,lon,lat,lon
  int position = comma3 + 1;
  for (byte i = 0; i < newCount; i++)
  {
    // Latitude start
    int commaLat = packet.indexOf(',', position);

    if (commaLat < 0)
    {
#if DEBUG
      Serial.println(F("Invalid latitude"));
#endif
      return;
    }

    String latText = packet.substring(position,commaLat);
    position = commaLat + 1;
    // Longitude start
    int commaLon = packet.indexOf(',', position);
    String lonText;

    if (commaLon >= 0)
```

```
    {
      lonText = packet.substring(position,commaLon);
      position = commaLon + 1;
    }
    else
    {
      // Last longitude
      lonText = packet.substring(position);
      lonText.trim();
      position = packet.length();
    }

    fence[i].lat = latText.toFloat();
    fence[i].lon = lonText.toFloat();
  }

  // Activate new fence
  fenceCount = newCount;
  // Force geofence state recalculation
  fenceStateKnown = false;
  sendCommandAck("FENCE", "OK");  // Tell base that fence was received

#if DEBUG
  Serial.println();
  Serial.println(F("================================"));
  Serial.println(F("FENCE UPDATED"));
  Serial.print(F("Animal: "));
  Serial.println(ANIMAL_ID);
  Serial.print(F("Points: "));
  Serial.println(fenceCount);

  for (byte i = 0; i < fenceCount; i++)
  {
    Serial.print(i);
    Serial.print(F(": "));
    Serial.print(fence[i].lat, 5);
    Serial.print(F(", "));
    Serial.println(fence[i].lon, 5);
  }

  Serial.println(F("================================"));
#endif
}
```

```
// RECEIVE LORA COMMAND
void receiveCommand()
{
  int packetSize = LoRa.parsePacket();
  if (packetSize <= 0)
  {
    return;
  }

  String packet = "";

  while (LoRa.available())
  {
    packet += (char)LoRa.read();
  }

  packet.trim();

#if DEBUG
  Serial.println();
  Serial.println(F("LoRa command received:"));
  Serial.println(packet);
#endif

  // MULTIHOP PACKET
  if (packet.startsWith("MH,"))
  {
    processMultiHopPacket(packet);
    return;
  }

  // ACK
  if (packet.startsWith("ACK,"))
  {
    processAlertAck(packet);
    return;
  }
  // POLL
  if (packet.startsWith("POLL,"))
  {
    receivePollCommand(packet);
    return;
  }
```

```
  // LIVE
  if (packet.startsWith("LIVE,"))
  {
    processLiveCommand(packet);
    return;
  }

  // FENCE
  if (packet.startsWith("FENCE,"))
  {
    processFenceCommand(packet);
    return;
  }

#if DEBUG
  Serial.println(F("Unknown LoRa command"));
#endif
}


// SETUP
void setup()
{
  //Serial.begin(9600);
  //packetSequence = 0;
  //savePacketSequence();
  loadPacketSequence();

  gpsSerial.begin(9600);
  Wire.begin();
  Serial.print(F("Animal: "));
  Serial.println(F(ANIMAL_ID));
#if DEBUG
  // Startup
  Serial.println();
  Serial.println(F("LIVESTOCK ANIMAL NODE"));

  // LoRa
  Serial.println(F("Starting LoRa..."));
#endif
  if (!LoRa.begin(LORA_FREQUENCY))
  {
    //Serial.println(F("LoRa FAILED"));
    while (1);
```

```
  }

  // LONG RANGE SETTINGS
  LoRa.setTxPower(18);
  LoRa.setSpreadingFactor(12);
  LoRa.setSignalBandwidth(125E3);
  LoRa.setCodingRate4(8);
  LoRa.setPreambleLength(12);
#if DEBUG
  Serial.println(F("LoRa OK"));
  Serial.println(F("TX Power: 18 dBm"));
  Serial.println(F("SF: 12"));
  Serial.println(F("BW: 125 kHz"));
  Serial.println(F("CR: 4/8"));
#endif

  // MPU
  //Serial.println(F("Starting MPU6050..."));

  if (!mpu.begin())
  {
    //Serial.println(F("MPU FAILED"));
    while (1);
  }

  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
  //Serial.println(F("MPU OK"));

  // Fence
  loadTestFence();
  delay(2000);

  //sendMultiHop("TEST,HELLO");

  // Battery
  readBattery();
  lastBatteryRead = millis();
#if DEBUG
  // Ready
  Serial.println(F("GPS ready"));
  Serial.println(F("Animal node ready"));
#endif
```

```
}

void loop()
{
  // GPS
  readGPS();
  if (pollWaitingForSlot && liveMode)
  {
    if (isMyDataSlotStart())
    {
      pollWaitingForSlot = false;

  #if PRINTE
      Serial.println(F("DATA SLOT START"));
      Serial.println(F("Sending telemetry..."));
  #endif

      sendTelemetry();
    }
  }

  //testAlertCommand();

  updateConditionalRelay();
  updateAlert();

  // MPU
  //updateMPU();

  updateBattery();

  // LoRa commands
  receiveCommand();

}

// PRINT BASIC STATUS
#if DEBUG
void printStatus()
{
  Serial.println();
  Serial.println(F("-------------------------"));
  Serial.print(F("Battery: "));
  Serial.print(batteryVoltage, 2);
```

```
Serial.println(F(" V"));
if (gps.location.isValid())
{
  Serial.print(F("Lat: "));
  Serial.println(gps.location.lat(), 5);

  Serial.print(F("Lon: "));
  Serial.println(gps.location.lng(), 5);

  Serial.print(F("Alt: "));
  Serial.println(gps.altitude.meters(), 1);

  Serial.print(F("Sat: "));
  Serial.println(gps.satellites.value());

  Serial.print(F("Speed: "));
  Serial.println(gps.speed.kmph(), 2);
}
Serial.print(F("Movement: "));

if (animalMoving)
{
  Serial.println(F("MOVING"));
}
else
{
  Serial.println(F("STATIONARY"));
}
Serial.print(F("Fence: "));
if (animalInside)
{
  Serial.println(F("INSIDE"));
}
else
{
  Serial.println(F("OUTSIDE"));
}

Serial.print(F("Live: "));
if (liveMode)
{
  Serial.println(F("ON"));
}
else
```

```
  {
    Serial.println(F("OFF"));
  }
  Serial.println(F("-------------------------"));
}
#endif

// READ GPS
void readGPS()
{
  while (gpsSerial.available())
  {
    if (gps.encode(gpsSerial.read()))
    {
      if (gps.location.isUpdated())
      {
        checkGeofence();
      }
    }
  }
}

// READ MPU6050
void readMPU()
{
  sensors_event_t a;
  sensors_event_t g;
  sensors_event_t temp;

  mpu.getEvent(&a, &g, &temp);

  float previousAcceleration = filteredAcceleration;

  float acceleration =
    sqrt(
      a.acceleration.x * a.acceleration.x +
      a.acceleration.y * a.acceleration.y +
      a.acceleration.z * a.acceleration.z
    );

  filteredAcceleration =
    FILTER_ALPHA * acceleration +
    (1.0 - FILTER_ALPHA) * filteredAcceleration;
```

```
  float accelerationChange =
    fabs(filteredAcceleration - previousAcceleration);

  float gyroMagnitude =
    sqrt(
      g.gyro.x * g.gyro.x +
      g.gyro.y * g.gyro.y +
      g.gyro.z * g.gyro.z
    );

  bool movementDetected =
    accelerationChange > ACCEL_THRESHOLD ||
    gyroMagnitude > GYRO_THRESHOLD;

  if (movementDetected)
  {
    if (movementCount < MOVEMENT_CONFIRM)
    {
      movementCount++;
    }

    stationaryCount = 0;

    if (movementCount >= MOVEMENT_CONFIRM)
    {
      animalMoving = true;
    }
  }
  else
  {
    if (stationaryCount < STATIONARY_CONFIRM)
    {
      stationaryCount++;
    }

    movementCount = 0;

    if (stationaryCount >= STATIONARY_CONFIRM)
    {
      animalMoving = false;
    }
  }
}
```

```
// READ BATTERY
void readBattery()
{
  int adc = analogRead(A0);
  //float voltage = adc * (5.0 / 1023.0);

  if (String(ANIMAL_ID) == "A01")
  {
    // A01 uses a voltage divider
    batteryVoltage = adc * (5.0 / 1023.0) * 2;
  }
  else
  {
    // Other nodes read battery directly
    batteryVoltage = adc * (5.0 / 1023.0);
  }
}

void printUTCDateTime(Print &out)
{
  if (!gps.date.isValid() || !gps.time.isValid())
  {
    out.print(F("0000-00-00,00:00:00"));
    return;
  }

  out.print(gps.date.year());
  out.print('-');

  if (gps.date.month() < 10)
    out.print('0');
  out.print(gps.date.month());

  out.print('-');

  if (gps.date.day() < 10)
    out.print('0');
  out.print(gps.date.day());

  out.print(',');

  if (gps.time.hour() < 10)
    out.print('0');
  out.print(gps.time.hour());
```

```
  out.print(':');

  if (gps.time.minute() < 10)
    out.print('0');
  out.print(gps.time.minute());

  out.print(':');

  if (gps.time.second() < 10)
    out.print('0');
  out.print(gps.time.second());
}
void updateMPU()
{
  if (millis() - lastMPURead < MPU_INTERVAL)
  {
    return;
  }

  lastMPURead = millis();

  readMPU();
}

void updateBattery()
{
  if (millis() - lastBatteryRead < BATTERY_INTERVAL)
  {
    return;
  }

  lastBatteryRead = millis();
  readBattery();
}

// LOAD TEST FENCE
// Temporary fence.
// Later ESP8266 will send this.
void loadTestFence()
{
  fenceCount = 4;
  fence[0].lat = 27.68600;
  fence[0].lon = 85.29500;
```

```
  fence[1].lat = 27.68800;
  fence[1].lon = 85.29500;

  fence[2].lat = 27.68800;
  fence[2].lon = 85.29800;

  fence[3].lat = 27.68600;
  fence[3].lon = 85.29800;

#if DEBUG
  Serial.print(F("Fence points: "));
  Serial.println(fenceCount);
#endif
}

void testAlertCommand()
{
#if DEBUG

  if (Serial.available())
  {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();

    if (cmd == "EXIT")
    {
      queueAlert(true);
      Serial.println(F("TEST EXIT ALERT TRIGGERED"));
    }
    else if (cmd == "RETURN")
    {
      queueAlert(false);
      Serial.println(F("TEST RETURN ALERT TRIGGERED"));
    }
  }

#endif
}

void loadPacketSequence()
{
  EEPROM.get(EEPROM_PACKET_SEQ_ADDR,packetSequence);
```

```
  if (packetSequence == 0xFFFFFFFFUL || packetSequence == 0)
  {
    packetSequence = 0;
  }
}

void savePacketSequence()
{
  EEPROM.put(EEPROM_PACKET_SEQ_ADDR,packetSequence);
}

// CHECK IF PACKET WAS ALREADY RECEIVED
bool isDuplicatePacket(String packetKey)
{
  for (byte i = 0; i < RECENT_PACKET_COUNT; i++)
  {
    if (recentPackets[i] == packetKey)
    {
      return true;
    }
  }
  return false;
}

// REMEMBER PACKET
void rememberPacket(String packetKey)
{
  recentPackets[recentPacketIndex] = packetKey;
  recentPacketIndex++;

  if (recentPacketIndex >= RECENT_PACKET_COUNT)
  {
    recentPacketIndex = 0;
  }
}

unsigned long getAlertBackoff()
{
  int animalNumber = 0;

  // ANIMAL_ID is like "A01", "A02", etc.
  if (ANIMAL_ID[0] == 'A')
  {
    animalNumber = atoi(ANIMAL_ID + 1);
```

```
  }

  // Safety
  if (animalNumber < 1)
  {
    animalNumber = 1;
  }

  unsigned long animalOffset = (unsigned long)(animalNumber - 1) * ALERT_BACKOFF_STEP;
  unsigned long randomOffset = random(0, ALERT_BACKOFF_RANDOM);
  return ALERT_BACKOFF_BASE + animalOffset + randomOffset;
}

// POINT IN POLYGON
bool insideFence(float lat, float lon)
{
  if (fenceCount < 3)
  {
    return false;
  }

  bool inside = false;
  byte j = fenceCount - 1;

  for (byte i = 0; i < fenceCount; i++)
  {
    float latI = fence[i].lat;
    float lonI = fence[i].lon;

    float latJ = fence[j].lat;
    float lonJ = fence[j].lon;

    if (
      ((lonI > lon) != (lonJ > lon)) &&
      (
        lat <
        (
          (latJ - latI) *
          (lon - lonI) /
          (lonJ - lonI) +
          latI
        )
      )
    )
```

```
    {
      inside = !inside;
    }
    j = i;
  }
  return inside;
}

// CHECK GEOFENCE
void checkGeofence()
{
  if (!gps.location.isValid())
  {
    return;
  }
  float lat = gps.location.lat();
  float lon = gps.location.lng();
  bool inside = insideFence(lat, lon);

  // First valid position
  if (!fenceStateKnown)
  {
    fenceStateKnown = true;
    animalInside = inside;

#if DEBUG
    Serial.println();
    Serial.println(F("INITIAL GEOFENCE"));
#endif
    if (inside)
    {
#if DEBUG
      Serial.println(F("Animal INSIDE"));
#endif
    }
    else
    {
#if DEBUG
      Serial.println(F("Animal OUTSIDE"));
#endif
      queueAlert(true);
    }
```

```
    return;
  }
  // State changed
  if (inside != animalInside)
  {
    animalInside = inside;
    if (inside)
    {
      queueAlert(false);
    }
    else
    {
      queueAlert(true);
      }
  }

}

// CHECK WHETHER COMMAND IS FOR THIS ANIMAL
// Example:
// LIVE,A02,ON
// For A02:
// returns true
// For A01:
// returns false
bool isForThisAnimal(const String &packet)
{
#if DEBUG
  Serial.println(F(">>> isForThisAnimal CALLED"));
  Serial.print(F("packet = ["));
  Serial.print(packet);
  Serial.println(F("]"));
#endif

  int firstComma = packet.indexOf(',');

  if (firstComma < 0)
  {
    return false;
  }

  int secondComma = packet.indexOf(',', firstComma + 1);

  if (secondComma < 0)
```

```
  {
    return false;
  }

  String targetID =
    packet.substring(firstComma + 1, secondComma);

  targetID.trim();

#if DEBUG
  Serial.print(F("targetID = ["));
  Serial.print(targetID);
  Serial.println(F("]"));

  Serial.print(F("ANIMAL_ID = ["));
  Serial.print(ANIMAL_ID);
  Serial.println(F("]"));
#endif

  return targetID == ANIMAL_ID;
}

unsigned long getGPSTimeSeconds()
{
  if (!gps.time.isValid())
    return 0;

  return (unsigned long)gps.time.hour() * 3600UL +
         (unsigned long)gps.time.minute() * 60UL +
         (unsigned long)gps.time.second();
}

bool isMyDataSlot()
{
  unsigned long gpsSeconds = getGPSTimeSeconds();

  if (gpsSeconds == 0)
    return false;

  unsigned long slot = gpsSeconds % GPS_SLOT_FRAME;

  if (ANIMAL_ID == "A01")
    return (slot >= A01_DATA_START &&
            slot <= A01_DATA_END);
```

```
  if (ANIMAL_ID == "A02")
    return (slot >= A02_DATA_START &&
            slot <= A02_DATA_END);

  return false;
}

bool isMyRelaySlot()
{
  unsigned long gpsSeconds = getGPSTimeSeconds();

  if (gpsSeconds == 0)
    return false;

  unsigned long slot = gpsSeconds % GPS_SLOT_FRAME;

  if (ANIMAL_ID == "A01")
    return (slot >= A01_RELAY_START &&
            slot <= A01_RELAY_END);

  if (ANIMAL_ID == "A02")
    return (slot >= A02_RELAY_START &&
            slot <= A02_RELAY_END);

  return false;
}

bool isMyDataSlotStart()
{
  unsigned long gpsSeconds = getGPSTimeSeconds();

  if (gpsSeconds == 0)
    return false;

  unsigned long frame = gpsSeconds / GPS_SLOT_FRAME;
  unsigned long slot = gpsSeconds % GPS_SLOT_FRAME;

  if (ANIMAL_ID == "A01")
  {
    if (slot >= A01_DATA_START &&
        slot <= A01_DATA_START + 1 &&
        frame != lastDataFrame)
    {
```

```
      lastDataFrame = frame;
      return true;
    }
  }

  if (ANIMAL_ID == "A02")
  {
    if (slot >= A02_DATA_START &&
        slot <= A02_DATA_START + 1 &&
        frame != lastDataFrame)
    {
      lastDataFrame = frame;
      return true;
    }
  }

  return false;
}

bool isMyRelaySlotStart()
{
  unsigned long gpsSeconds = getGPSTimeSeconds();

  if (gpsSeconds == 0)
    return false;

  unsigned long frame = gpsSeconds / GPS_SLOT_FRAME;
  unsigned long slot = gpsSeconds % GPS_SLOT_FRAME;

  if (ANIMAL_ID == "A01")
  {
    if (slot >= A01_RELAY_START &&
        slot <= A01_RELAY_START + 1 &&
        frame != lastRelayFrame)
    {
      lastRelayFrame = frame;
      return true;
    }
  }

  if (ANIMAL_ID == "A02")
  {
    if (slot >= A02_RELAY_START &&
        slot <= A02_RELAY_START + 1 &&
```

```
      frame != lastRelayFrame)
    {
     lastRelayFrame = frame;
     return true;
    }
  }

  return false;
}
```